- *Static analysis tools*—These check the logic in your code and alert on common fail points and security weaknesses. Popular Python static analysis tools are SonarQube (https://www.sonarqube.org/) and Codacy (http://codacy.com/). Note that both of these are commercial products with free and open source plans.

... not as I do

When writing the book, I added new features to RiffMates as I went along, chapter by chapter. As testing wasn't covered until this chapter, I was lazy and didn't write unit tests for the project. This is bad practice, and it bit me. I found all sorts of problems in the sample code when I started writing tests. Writing automated tests should be treated as part of coding, not an "after task," as not having them piles up and becomes problematic.

All the code in this chapter is in RiffMates/bands/tests.py, but that doesn't give good coverage of the project—it doesn't even cover all the views in the Bands app. To help catch problems, the sample code also contains RiffMates/tests/test_all.py. The name is a bit misleading, as it doesn't quite test everything, but it brings the coverage number up above 80%, which is a common threshold for being "good enough."

8.9 Exercises

1. In chapter 7's exercises, you added a view called `edit_musician()`, where users could create and manage their own musician profile page. Add a test for that view. Include testing for fetching the form, submitting the form, and adding a musician with a picture. A user can edit the musician profile they created, and so can staff and superusers. Test that those types of users can edit, while a non-owner cannot. Note that you can create a superuser with the `User.objects.create_superuser()` method.
2. Write a test for the `musicians()` view that validates that pagination is working. The view needs to be called multiple times with a variety of query parameters to test different settings for the number of items per page as well as the next and previous arguments. Note that the paginator's `.has_previous()` and `.has_next()` are methods, not attributes; you didn't need the parenthesis in the template, but you do when calling them in code.

Summary

- Automated testing is typically approached hierarchically, with each level of tests having a separate purpose and possibly approach.
- A unit test is the lowest level in the hierarchy and is performed on a single component of software, typically a function or class method.
- Integration tests verify that components interact well together and operate across several units.